

Day 08 – SQL Injection Beginner Lab

Trios Cyber Internship

Assignment: SQL Injection Beginner Lab

Day: 08

Application: Damn Vulnerable Web Application (DVWA)

Target: <http://192.168.56.101/dvwa/>

Environment: Authorized local VirtualBox laboratory

Tools: Burp Suite, Firefox

Testing Level: Beginner / Low Security

1. Objective

The objective of this practical was to complete two beginner-level SQL injection exercises against the intentionally vulnerable DVWA application.

The assessment focused on identifying user-controlled parameters, capturing HTTP requests, modifying the relevant parameter in Burp Suite Repeater, and comparing the resulting application responses.

2. Laboratory Environment

Component	Details
Attacker	Kali Linux
Target	DVWA
Target IP	192.168.56.101
Application URL	http://192.168.56.101/dvwa/
Virtualization	Oracle VirtualBox
Proxy Tool	Burp Suite
Browser	Firefox
Security Level	Low
Scope	Authorized local laboratory

3. SQL Injection Overview

SQL Injection is a web application vulnerability that occurs when untrusted user input is incorporated into SQL queries without appropriate validation or parameterization.

A vulnerable application may allow specially crafted input to alter the intended logic of a database query.

In this practical, DVWA was intentionally used because it provides controlled SQL injection training functionality.

4. Exercise 1 – SQL Injection

Vulnerable Functionality

The first exercise used the DVWA SQL Injection functionality.

The application accepts an `id` parameter from the user and uses it to retrieve database records.

Vulnerable Parameter

“text id

Evidence

Exercise 1 – SQL Injection

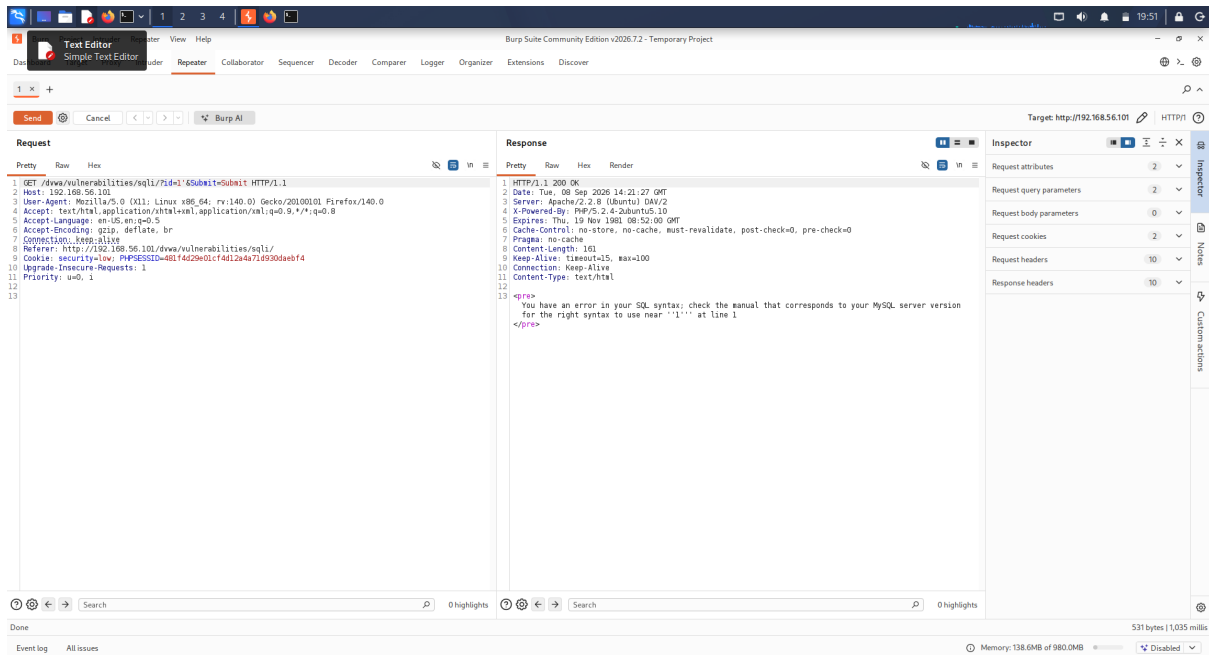


Figure 1: SQL Injection Exercise 1

Exercise 2 – Additional SQL Injection Repeater Test

Exercise 3 – Blind SQL Injection

Scope and Ethics

All testing was performed against the intentionally vulnerable DVWA application hosted inside the authorized local VirtualBox laboratory environment.

No external systems were targeted.

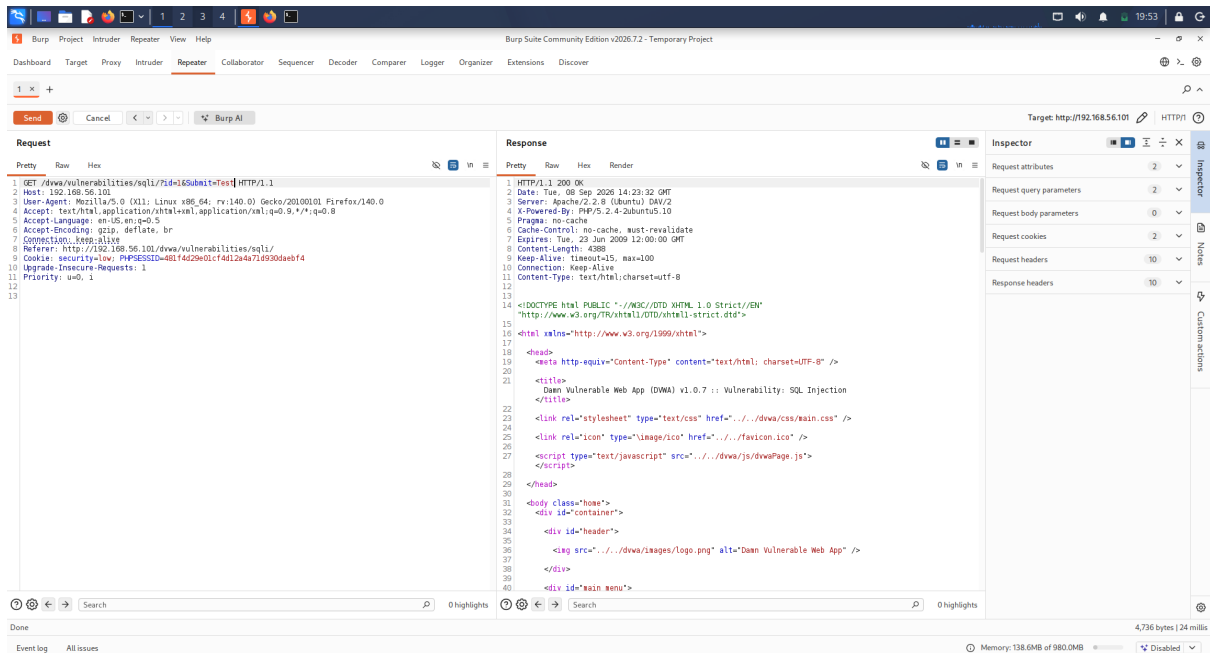


Figure 2: SQL Injection Exercise 2

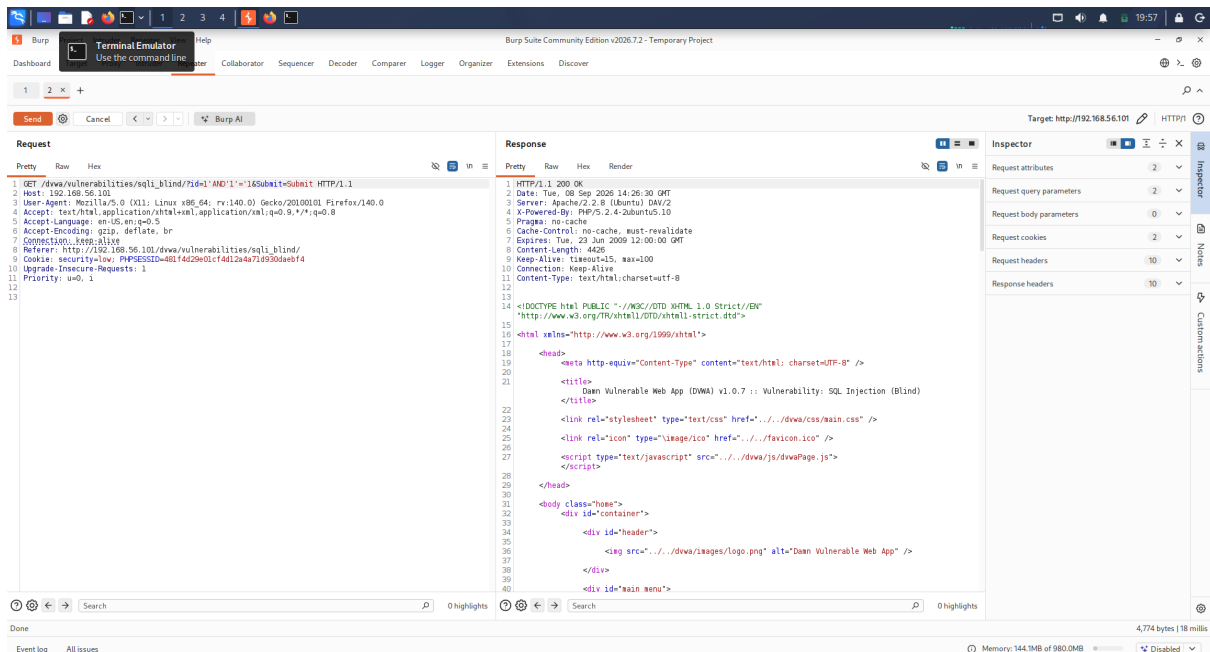


Figure 3: Blind SQL Injection Exercise